

REST API with Node.js + Express.js

POST

GET

Presentation overview

1 Introduction

What REST, Node.js, and Express.js are

2 Code example

Run through a simple Node.js + Express.js app

3 Application to capstone

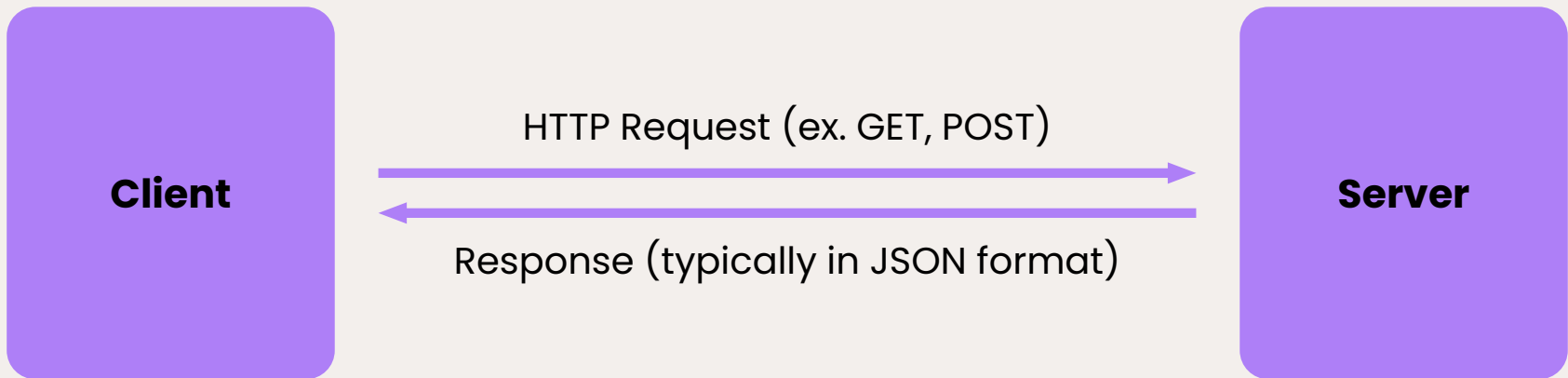
How you may find these technologies useful

4 Wrap up

Summary of key points and takeaways

REST

- REST stands for “***Representational State Transfer***”
 - ie. how a server transforms and sends data to your device
- Architectural style that describes how APIs should be designed for communication between client and server over HTTP



Architecture constraints

- In order to be considered “**RESTful**”...
 - **Separation of client/server concerns.** Client is user-facing while the server handles data and business logic
 - **Server is stateless** and shouldn't rely on storing context from previous requests
 - **Layered approach.** Client does not care how about how a resource is returned, only cares about making a request to the API
 - **Uniform interface** to create, access and retrieve resources

REST API

1. Resource

Tasks

+

2. URI (Uniform Resource Identifier)

/Tasks/1

+

3. HTTP Method

GET
POST
PUT
DELETE

...

Node.js

- JavaScript runtime environment for servers
- Provided developers the opportunity to write full-stack applications in JavaScript
- Initial release in 2009

Express.js

- Popular Node.js web framework that simplifies the process of building RESTful applications
- Provides beneficial abstractions that help keep large, complex apps maintainable

Code example

```
// Import the Express module
const express = require('express');
const app = express();
const port = 3000;

app.use(express.json());

...

// Start the server
app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});
```

Code example

...

// Define a route for GET requests

```
app.get('/users', (req, res) => {  
  const users = [ { id: 1, name: "Alice" }, { id: 2, name: "Bob" } ];  
  res.json({ message: 'Returning list of users', users: users });  
});
```

// Define a route for POST requests

```
app.post('/users', (req, res) => {  
  const newUser = req.body;  
  ...  
  res.status(201).json({ message: 'User created', user: newUser });  
});
```

...

Application to capstone

Frequently used in web applications

- 1 Technologies such as Express.js provide support for abstracting away complexities, making it an enticing choice for those building web applications. You're likely to encounter it sometime.

Instills best practice

- 2 REST is a developed standard in web development. Best practices have already been proven to be effective, providing a roadmap for how to build scalable, maintainable software.

Think smarter, not harder

- 3 Instead of immediately creating a custom solution, check if someone else has already built one. This could save you time and energy that you could put towards solving other problems.

Wrap up

- **REST:**
 - Architectural style for managing resource access, creation and retrieval between client and server by HTTP methods
- **Node.js:**
 - JavaScript runtime environment that doesn't require a browser
- **Express.js:**
 - Web framework built on top of Node.js that helps simplify building RESTful applications

Thanks!

**GeeksforGeeks. "Rest Api Introduction." *GeeksforGeeks*, 8 July 2026,
www.geeksforgeeks.org/node-js/rest-api-introduction/.**

**Fireship. "Node.Js Ultimate Beginner's Guide in 7 Easy Steps." *YouTube*, 2020,
www.youtube.com/watch?v=ENrzD9HAZK4.**